CE-355 Lab 4: Mandelbrot Set Explorer
Team 16: Matias Ketema, Albert Luo

Simulation

We simulated the design in ModelSim using the provided testbench, which uses 16 engines, MAX_ITER of 62, PIX_DIV of 4 and ZOOM_NUM of 15. It starts in Seahorse Valley and writes eight BMP images, 12 frames apart. Figure 1 shows the transcript. All files compiled, each image was written, and no assertion fired, including the bank's check that an engine is never read before it finishes.

```
# Loading work.mandel_engine(rtl)
# ** Note: Drawing image 1 of 8
#    Time: 2585 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote image_1.bmp
#    Time: 12290585 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Drawing image 2 of 8
#    Time: 12321425 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote image_2.bmp
#    Time: 24609425 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Drawing image 3 of 8
#    Time: 24640265 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote image_3.bmp
#    Time: 36928265 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Drawing image 4 of 8
#    Time: 36959105 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote image_4.bmp
#    Time: 49247105 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Drawing image 5 of 8
#    Time: 49277945 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote image_5.bmp
#    Time: 61565945 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Drawing image 6 of 8
#    Time: 61596785 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote image_6.bmp
#    Time: 73884785 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Drawing image 7 of 8
#    Time: 73915625 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote image_7.bmp
#    Time: 86203625 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Drawing image 8 of 8
#    Time: 86234465 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote image_8.bmp
#    Time: 98522465 ns  Iteration: 1  Instance: /mandelbrot_tb
# ** Note: Wrote 8 images, 12 frames apart
#    Time: 98550735 ns  Iteration: 1  Instance: /mandelbrot_tb
```

Figure 1 ModelSim transcript

Figure 2 shows the rendered output. Each image has a black region with smooth colored bands around it, and each one is zoomed further into the same point than the last. Smooth bands show that the signed casts and the fixed-point slices are correct, because errors in those produce noise. A straight, unsheared picture shows that pixels come back from the engines in the right order.

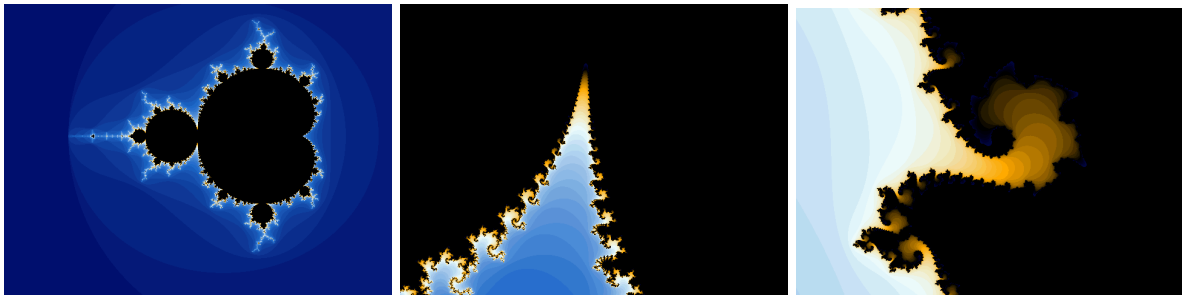


Figure 2: Simulation images output (just images 1, 4, and 8 to save space)

Figure 3 shows engine 0 computing one point. After the one-clock start pulse, clock_count runs from 0 to MAX_ITER, and x and y change on each clock until escaped goes high. They then hold their values, and escape_count stops at the same time. The counter keeps running, and ready goes high at the end with the final color on pixel. This confirms that every pixel takes the same number of clocks whether it escapes early or not.

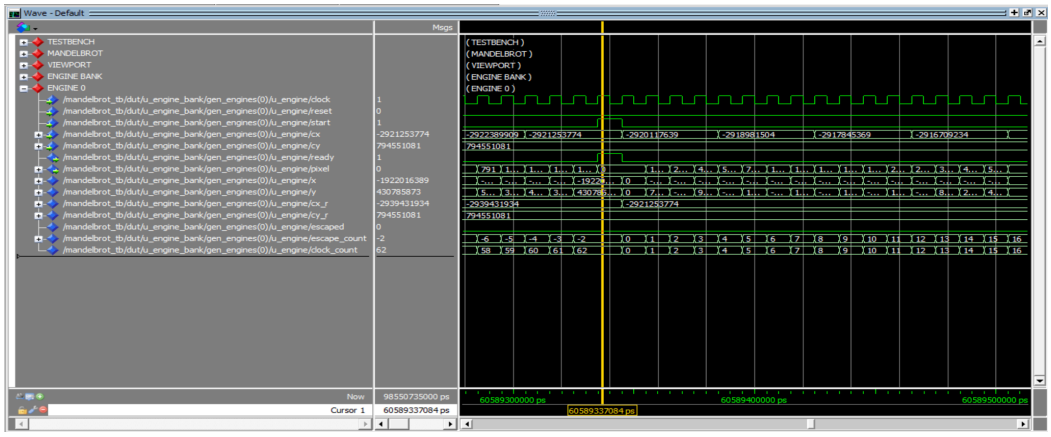


Figure 3

Figure 4 shows the head start in the gap between frames. The viewport updates x0, y0 and pixel_step when frame_sync rises. The bank loads the new corner one clock later and then starts one engine per clock until the first 16 points are handed out. No pixel_req is active during this time.

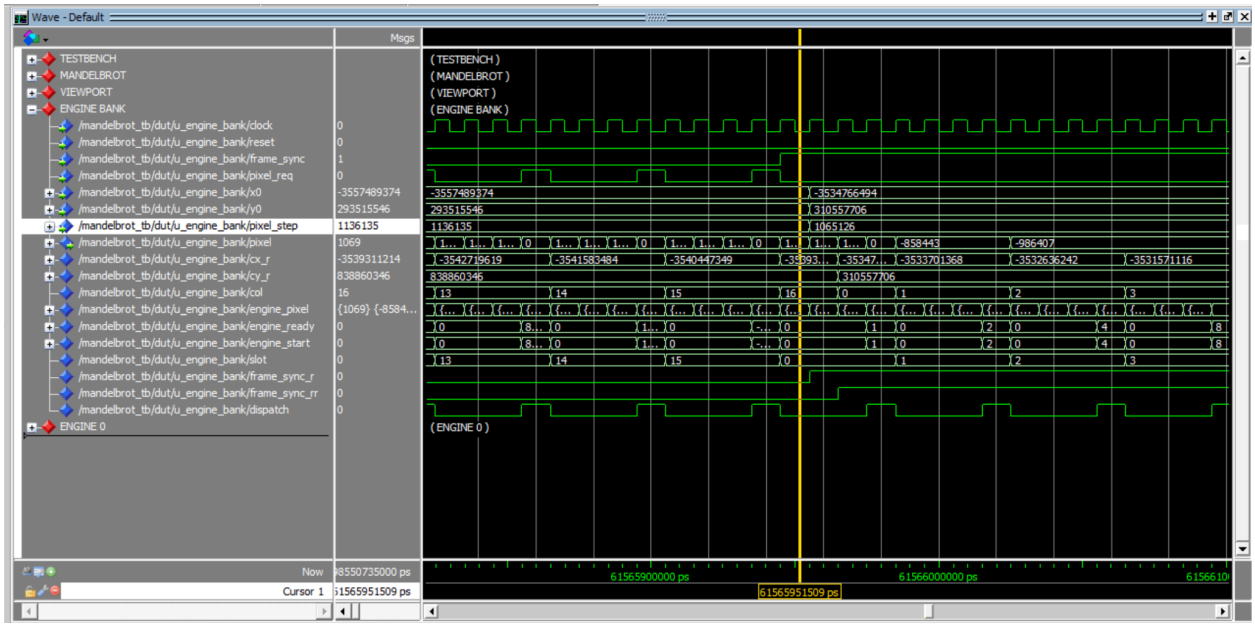


Figure 4

Schematic

Figure 5 shows the RTL Viewer output. The top level connects VIDEO_PLL, mandelbrot and hdmi, and all three share the single 100 MHz clock. Inside mandelbrot, u_viewport drives x0, y0 and pixel_step into u_engine_bank, which holds the gen_engines array of 28 mandel_engine

instances. Each engine has three signed multipliers and its own palette lookup.

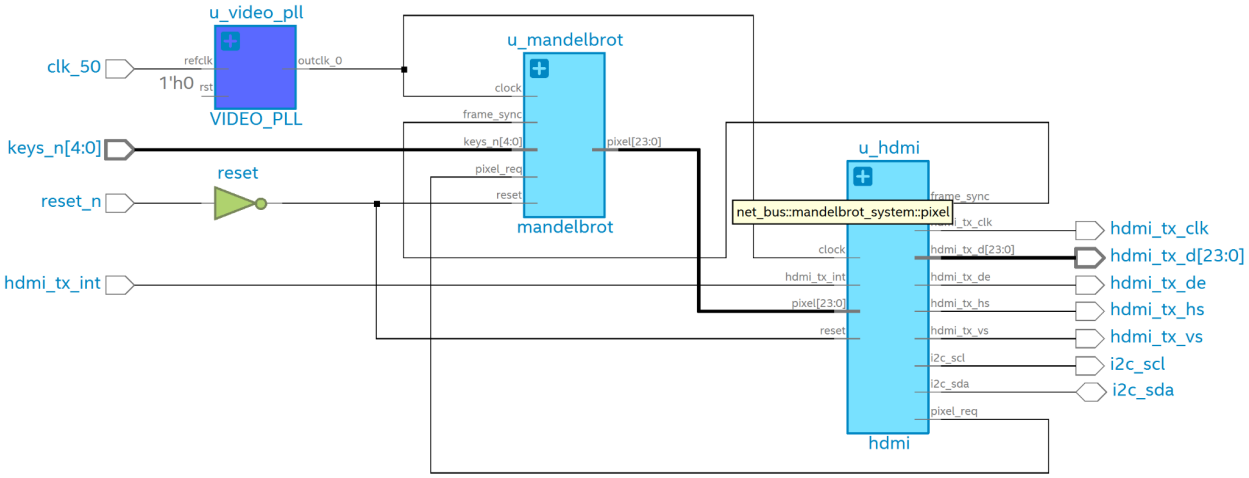


Figure 5

Resources

We expected about 6 DSP blocks per engine, or 168 for 28 engines, and the Fitter reported 168 out of 188, which matches exactly. This means the viewport's constant multiplies did not use any DSP blocks. We estimated about 5,500 registers: about 184 per engine, or about 5,150 in total, plus about 300 for the bank, viewport and hdmi. The Fitter reported 3,586. The difference is most likely because Quartus packed the x and y input registers into the DSP blocks, which is up to 72 bits for each of the 28 engines, or about 2,000 registers. The design has no frame buffer, so we expected no memory for image storage.

Timing

The design has a 50 MHz input clock and a 100 MHz PLL output clock. The 50 MHz clock only feeds the PLL, so it has no register paths and no fMAX or setup slack is reported for it. Only the 100 MHz clock drives the logic. Its fMAX is 100.62 MHz with a worst-case setup slack of 0.062 ns, so the design meets timing with very little margin, using timing models that Quartus marks as preliminary. The critical path runs from the input register of a multiplier DSP block in engine 25 (mult_1~mac_2~AY_REG) to x[32]~_Duplicate3, a copy of the x register that the fitter placed at the input of another DSP block (mult_2~mac_2). The data path delay is 9.71 ns out of the 10 ns clock period, and 3.98 ns of that is the DSP block itself. The rest is routing (3.06 ns) and logic (2.66 ns). The path goes from the multiplier output through more carry-chain logic in mult_1, then through the two adders that compute the new x, and into the register. This is the slowest path because one iteration must finish in one clock, so the 36 by 36 bit multiply and the update adders all sit between two registers with no pipeline stage. The viewport arithmetic is also slow, but it has a multicycle constraint because those registers change only once per frame. We did not need design changes to meet timing.

Hardware and Attribution

The hardware tests were: full set centered at reset, panning in all four directions, new detail appearing while zooming in and zooming back out with SW0, reset returning to the starting view, and the picture returning after reconnecting the HDMI cable, in which all the tests passed. The hardware uses 28 engines, MAX_ITER of 110 and ZOOM_NUM of 255, while the testbench uses 16, 62 and 15, so the board shows more detail and zooms more slowly. The testbench also generates pixel_req and frame_sync itself instead of using hdmi. One other issue we have is the polarity of our switch seems to be flipped. So it zooms in on boot up when the switch is not flipped.